总体概览

核心区别可以概括为：新的写法（Composition API）、更快的性能、更好的 TypeScript 支持，以及部分 API 变更/移除。

---

1. 编程范式：Options API vs Composition API

- **Vue 2：Options API 为主**
  - 通过 `data / methods / computed / watch / created / mounted` 等选项来组织代码。
  - 逻辑容易“分散”：一个功能的相关代码可能分布在多个选项块中。
- **Vue 3：新增 Composition API（组合式 API）**
  - 使用 `setup()` 、 `ref` 、 `reactive` 、 `computed` 、 `watch` 等来组织逻辑。
  - 可以把同一业务逻辑聚合到一起，方便抽离为可复用的 `useXxx` 函数（类似 React Hooks）。
  - Options API 仍然支持，但官方更推荐在新项目中用 Composition API。

---

2. 响应式原理： `Object.defineProperty` vs `Proxy`

- **Vue 2**
  - 基于 `Object.defineProperty` 劫持属性。
  - 缺点：无法监听属性的新增/删除，对数组的某些变更需要特殊处理（如 `Vue.set` 、 `this.$set` ）。
- **Vue 3**
  - 基于 ES6 `Proxy` 实现响应式。
  - 优点：可监听对象属性的新增/删除、数组下标、 `Map/Set` 等复杂结构；整体性能更好。
  - API 层面体现为： `reactive()` 、 `readonly()` 、 `shallowReactive()` 等。

---

3. 性能 & 打包优化

- **编译与运行性能**
  - Vue 3 进行了编译器和运行时的全面重写，渲染性能更好，内存占用更低。
  - 静态提升、Patch Flag 等编译优化，减少运行时的 diff 工作量。
- **Tree-shaking 友好**
  - Vue 3 采用模块化设计（ `@vue/runtime-core` 、 `@vue/runtime-dom` 等）。
  - 按需导入 API（如 `import { ref, reactive } from 'vue'` ），未使用的代码能被打包工具更好地 Tree-shaking 掉，产物更小。

---

4. TypeScript 支持

- **Vue 2**
  - TS 支持相对“补丁式”，对类型推导不够友好，复杂项目里经常要写很多声明。

- **Vue 3**
  - 从设计之初就考虑 TypeScript，源码本身用 TypeScript 编写。
  - Composition API 更符合 TS 的思路，类型推导和提示都更强。

---

5. 生命周期钩子变更

Vue 3 在 Composition API 中使用新的钩子名（Options API 基本沿用旧名）：

- `beforeCreate` / `created` → 用 `setup()` 代替。
- `beforeMount` → `onBeforeMount`
- `mounted` → `onMounted`
- `beforeUpdate` → `onBeforeUpdate`
- `updated` → `onUpdated`
- `beforeDestroy` / `destroyed` → `onBeforeUnmount` / `onUnmounted`
- `activated` / `deactivated` → `onActivated` / `onDeactivated`
- `errorCaptured` → `onErrorCaptured`

---

6. 全局 API 与应用实例

- **Vue 2**
  - 全局 API 挂在 `Vue` 构造函数上，如 `Vue.use`、`Vue.mixin`、`Vue.component` 等。
  - 创建根实例：`new Vue({ el: '#app', ... })`
- **Vue 3**
  - 使用 `createApp` 创建应用实例：

```
import { createApp } from 'vue'
import App from './App.vue'

const app = createApp(App)
app.mount('#app')

```

  - 全局 API 都通过 `app` 实例来调用：
    - `app.use(...)`
    - `app.component(...)`
    - `app.directive(...)`
  - 支持多应用实例共存，更灵活。

---

7. 模板语法与常用特性差异

- **`v-model` 改进**
  - Vue 2：`v-model` 默认绑定 `value` + `input` 事件，自定义组件需要 `value` / `input` 。
  - Vue 3：`v-model` 默认绑定 `modelValue` + `update:modelValue` 事件。
  - 支持多个 `v-model` ： `v-model:title="title"` 对应 `title` / `update:title` 。
- **过滤器（filter）被移除**
  - Vue 2 有 `{{ price | currency }}` 这种模板过滤器。
  - Vue 3 移除了 filter，推荐使用 `computed` 或普通函数、管道函数链等方式处理显示数据。
- **Fragments / Teleport / Suspense**
  - Vue 2：组件模板必须有单根节点。
  - Vue 3：引入 Fragment，一个组件可以返回多个根节点。
  - `Teleport` ：可以把子组件内容渲染到 DOM 树的别处（比如全局的 `body` ）。
  - `Suspense` ：用于更好地处理异步组件渲染（实验特性）。

---

8. 生态与周边库

- **路由**
  - Vue 2 搭配 `vue-router@3`
  - Vue 3 搭配 `vue-router@4` （API 有小变更，比如 `createRouter` 、 `createWebHistory` 等）
- **状态管理**
  - Vue 2 时代主流是 `vuex` 。
  - Vue 3 时代官方推荐使用 `pinia` （更轻量、TS 友好）。

---

9. 兼容性与迁移

- Vue 官方提供了 **迁移指南**，以及 **兼容构建版本（@vue/compat）** ，帮助从 Vue 2 逐步迁移到 Vue 3。
- Vue 3 在设计时尽量保持大部分常用用法不变，但部分 API（如 filter、一些实例方法 `$on/$off/$once` 等）被废弃或调整。